

ArCHer 中文精读导读：用“分层强化学习”教语言模型当多轮智能体

原文：Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine (UC Berkeley, 加州大学伯克利分校)
Aviral Kumar (Google DeepMind, 谷歌深度思维)

本导读为面向 AI 应用开发初学者的中文学习笔记：含要点讲解、关键句翻译、公式精讲与图表解读。出于版权合规，这是转化性学习材料，以转述和讲解为主，不逐字翻译原文。

论文卡片

英文标题	ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL (ArCHer: 用分层式多轮强化学习训练语言模型智能体)
作者	Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, Aviral Kumar
机构	UC Berkeley (加州大学伯克利分校)、Google DeepMind (谷歌深度思维)
会议	ICML 2024 (国际机器学习大会)
引用数	180 (Semantic Scholar, 截至 2026-06-07)
代码	https://github.com/YifeiZhou02/ArCHer
方向	多轮智能体 (multi-turn agent) × 大模型 × 强化学习 (RL) 后训练
一句话定位	用分层 <i>actor-critic</i> (演员-评论家) 框架解决多轮长程任务里的“功劳分配” 难题：高层用 off-policy 的 TD 学习训一个话语级价值函数，评估“说完这句话局面好坏”；低层用 on-policy 的 <i>token</i> 级策略梯度决定这句话每个字怎么生成。 在 Twenty Questions 等任务上，样本效率比纯 on-policy 的 PPO 约提升 100 倍。

1 小白知识卡：读这篇前要懂的几个概念

这篇论文站在“大模型 + 强化学习 + 智能体”的交叉点上，下面这几个词先弄明白，后面就顺了。

- **智能体任务 (agent task) / 多轮 (multi-turn)**：让大模型不止“回答一个问题就完事”，而是要连续做好几步决策去完成一个目标。比如：在网页上一步步操作买东西、在 Linux 终端里先读手册再敲命令、跟用户来回聊好几轮把需求问清楚。每一步叫一“轮” (turn)，每一轮模型说的那句话叫一个话语 (utterance)。

- **强化学习 (reinforcement learning, RL)**: 一种“试错学习”。智能体在环境里行动 (action), 环境给反馈 (状态 state 变化 + 奖励 reward), 目标是让长期累计奖励最大。和监督学习不同, 它不需要“标准答案”, 只需要一个能打分的奖励信号。
- **单轮 RL vs. 多轮 RL**: 现在主流的 RLHF (基于人类反馈的强化学习) 大多是单轮的——给一个 prompt、生成一个回答、用奖励模型打个分、更新一下, 就结束了。它不需要模型考虑“我这句话会怎么影响后面几轮”。多轮 RL 则必须考虑这种长期影响。
- **PPO (近端策略优化, Proximal Policy Optimization)**: 当前 RLHF 最常用的算法, 属于 *on-policy* (同策略) 方法——每次更新只能用“当前这版模型刚刚跑出来的数据”, 旧数据用完就扔。优点是稳, 缺点是样本效率低 (很费数据/很费与环境交互的次数)。
- **on-policy (同策略) vs. off-policy (异策略)**: on-policy 只能用当前策略产生的数据; off-policy 可以把过去所有交互存进一个回放缓冲区 (replay buffer) 反复利用。在多轮任务里, 每次与环境交互 (比如真的去点一次网页) 又慢又贵, 所以能复用旧数据的 off-policy 方法在样本效率上有巨大优势。
- **功劳分配 / 信用分配 (credit assignment)**: 长程任务的核心难题。一连串动作最后才拿到奖励, 到底是哪一步的功劳 (或过错)? 比如二十问游戏里赢了, 是因为第 3 个问题问得好, 还是第 15 个? 把奖励正确地“摊派”回每一步, 就是 credit assignment。
- **actor-critic (演员-评论家)**: 一类 RL 算法。*actor* (演员) = 策略, 负责“做什么动作”; *critic* (评论家) = 价值函数, 负责“评估当前局面/动作有多好”。critic 给 actor 提供更稳的学习信号, 减少噪声。
- **价值函数 / Q 函数 (value / Q function) 与 TD 学习**: $V(s)$ 估计“处在状态 s 时, 未来能拿多少累计奖励”; $Q(s, a)$ 估计“在状态 s 做动作 a 后能拿多少”。TD 学习 (temporal-difference, 时序差分) 是训练这些函数的经典办法: 用“当前奖励 + 下一步的估值”来纠正“当前的估值”, 即所谓 Bellman 自举 (bootstrapping), 不必等到一整局结束才学。
- **token 级 vs. 话语级 (token-level vs. utterance-level)**: 把模型生成的每一个字 (*token*) 都当成一个动作, 叫 token 级; 把一整句话当成一个动作, 叫话语级。token 级动作空间小、好优化, 但一局下来 token 数量巨多、时间跨度 (horizon) 极长, 奖励信号要传播很久; 话语级时间跨度短, 但“一句话”的动作空间是天文数字 (所有可能的句子), TD 学习里要“在所有动作里取最大值”变得不可行。ArCHer 的巧思就是两者各取所长。

【小白补丁】一句话抓住这篇在干嘛

多轮智能体训练, token 级 RL “时间跨度太长、学得太慢”, 话语级 RL “动作空间太大、没法取最大值”, 纯 on-policy 的 PPO 又“太费数据”。ArCHer 把它们分两层: 高层把“一句话”当一个动作, 用能复用旧数据的 off-policy TD 学习训一个评委 (话语级 critic); 低层把“一句话里的每个字”当动

作，用 PPO 那套 on-policy 策略梯度生成，奖励就用高层评委给的分。于是同时拿到“短跨度 + 小动作空间 + 高样本效率”。

2 摘要全译

大语言模型 (large language models, LLMs) 的一大类用途是面向目标的决策任务 (即“智能体”任务)：模型不只是给定 prompt 生成一个像样的续写，而是要在多轮交互中、在较长时间跨度内做出一连串聪明决策来完成任务 (例如与网页交互、使用软件工具、做客服支持)。强化学习 (RL) 为这类智能体任务提供了通用范式，但当前面向 LLM 的 RL 方法大多聚焦于单轮的奖励最大化。从设计上看，今天的单轮 RL 方法根本无法训练 LLM 去“跨多轮地主动搜集并整合信息”、做“功劳分配”、或对自己过去的动作进行推理——而这些恰恰是智能体任务的关键。这就提出一个问题：我们该如何为 LLM 设计有效又高效的多轮 RL 算法？

本文提出一个用于开发“微调 LLM 的多轮 RL 算法”的算法框架，它既保留了现有单轮 RL 方法 (如 PPO) 的灵活性，又能有效地容纳多轮、长时间跨度和延迟奖励。为此，我们的框架采用分层 RL (hierarchical RL) 思路，并行运行两个 RL 算法：一个高层 *off-policy* 的 RL 算法，训练一个价值函数来在话语层面聚合奖励；一个低层 RL 算法，利用这个高层价值函数 (取代单轮 RL 里用的奖励模型) 来逐字 (token-by-token) 训练每一轮/每句话内部的策略。我们把这个框架命名为 **ArCHer** (Actor-Critic Framework with a Hierarchical Structure, 带分层结构的演员-评论家框架)，它还能衍生出其他若干种 RL 方法。实验上我们发现，ArCHer 在多轮任务上显著提升了效率与性能：相比现有 on-policy 方法，样本效率提升约 **100 倍**；同时它还能很好地受益于模型容量扩大 (我们测到了 70 亿参数规模)。项目主页：<https://yifeizhou02.github.io/archer.io/>，代码：<https://github.com/YifeiZhou02/ArCHer>。

3 逐节精读

3.1 引言：为什么单轮 RL 教不出好智能体

LLM 凭借通用知识，有潜力解决各种能用自然语言表达的决策/“智能体”问题：写代码、上网操作、用工具、与人交流等等。要在这些场景里成功，模型必须跨多轮做出一连串聪明决策，而不是每步只生成“概率最高的续写”。

但现状是：大多数让 LLM 表现出目标导向行为的方法，都依赖短视的目标——要么逐步模仿成功示范 (行为克隆)，要么优化单轮偏好 (如 RLHF)。作者点出单轮方法的两个硬伤 (这两句是全文动机的关键)：

“用单轮方法训练出来的策略，往往做不好功劳分配 (例如，无法识别出那些当下看似次优、却能带来长期收益的好动作)，也无法赋予策略主动搜集信息的行为——而这在智能体问题中很重要 (例如面对一个新工具时)。”

图 1 的对比直觉：原文图 1 用一个“用户说没动力工作”的例子做对比。单轮智能体试图在一轮里把问题解决，于是一股脑甩出一大堆建议（结果用户觉得是陈词滥调，不满意）；多轮智能体则先反问“最近是不是发生了什么事”，先搜集信息再针对性回应，最后用户很满意。技术上，单轮 RL 用“奖励模型 + 策略梯度”训练；本文的多轮做法是把奖励模型换成一个用 *off-policy RL* 学出来的价值函数。

多轮 RL 的独特挑战（作者归纳）：

1. 多轮 RL 需要与外部环境（人、网页服务器）在线交互，又慢又贵。因此像 PPO 这种 on-policy 方法很快变得不实用——它不能复用过去的交互数据。
2. off-policy 的纯离线方法能复用数据，但又有新麻烦：多轮下 token 数量暴涨（LLM 还偏爱长回答），把每个 token 当动作的方法（如 token 级 ILQL）就得在极长的时间跨度上传播奖励，学得极慢。作者实测：token 级 ILQL 在某任务上要 10 天以上才收敛，而过滤式行为克隆不到一天。
3. 反过来，把一整句话当一个动作能缩短时间跨度，但会带来一个巨大且变长的动作空间——这对基于 TD 学习、每步都要“在动作上取最大值”的 off-policy 方法是灾难。

核心洞见：用分层的办法同时绕开上面所有坑。高层用 off-policy TD 学习训一个话语级价值函数；低层用任意 on-policy 策略梯度优化每一轮内部的 token 生成，并把高层价值函数当作“这一轮结束时的奖励”。这样既能复用样本、收敛快，又避免了“对单个 token 做 Bellman 回溯”或“在天文数字的动作空间里取最大值”。一句话总结贡献：

“这样我们就同时拿到了话语级与 token 级、off-policy 与 on-policy 各自的好处。”

3.2 相关工作：前人卡在哪

- **LLM 的单轮 RL：**PPO、A2C、DPO、过滤式监督学习等，都聚焦“模型只做一个决策、不再与环境交互”的单轮偏好优化。但很多智能体问题无法在单轮内解决——它们需要先主动搜集信息再决策（如先问清用户偏好再订行程、先读终端帮助手册再执行命令），单轮方法学不会这种细腻策略。
- **不用 RL 训练语言智能体：**靠提示工程（prompt engineering），如 ReAct（先“想”再做）、Reflexion（反思过去失败）、Voyager（自建课程与动作库）。但不更新模型参数，效果天花板被预训练能力锁死；连 GPT-4 加上下文学习，在分布外场景也可能表现很差。另一条路是用成功轨迹微调，但人工标注/工具调用标注很贵，且任务越长，自动“撞”到成功轨迹的概率越低、代价越高。
- **LLM 的多轮 RL：**直接套用 PPO、REINFORCE 等策略梯度，在需要与环境交互的多步场景里样本效率很低。off-policy / 离线的价值方法能用静态数据学，但现有方法要么 (1) 把单个 token 当动作（token 级），被超长时间跨度拖累；要么 (2) 把一句话当动作，却得从一个冻结的预训练 LLM 里采样多个候选句子来做 Bellman 回溯里的“取最大值”，这会拖慢策略改进的速度。ArCHer 要同时解决这两个局限。

3.3 方法第一步：把语言生成建模成“分层 MDP”

这是全文方法的地基。作者把多轮语言生成重新形式化为在一个分层马尔可夫决策过程 (hierarchical Markov decision process, MDP) 里行动——一个高层 MDP，里面嵌套一个低层 MDP。

- 两层 MDP 的状态都是变长的 *token* 序列。
- 高层 MDP：一个动作 a_t 是一整段 *token* 序列（一句话/一轮）；状态 s_t 是“LLM 与环境到目前为止的交互历史”。
- 低层 MDP：一个动作 a_t^h 是单个 *token*（第 t 句话里的第 h 个字）。低层 MDP 建模的是“怎么把高层那一句话一个字一个字生成出来”。低层的状态 = 拼接“到这一轮为止的高层状态 s_c ” + “本轮内已经生成的字 $a_t^{1:h-1}$ ”。
- 两层如何衔接：在低层 MDP 里执行一连串动作（吐出一串 *token*），恰好等于在高层 MDP 里执行一个动作（说完一句话）。低层何时结束？当策略选择吐出“EOS”（结束符）时。

两层各自的目标（关键）：高层策略最大化任务奖励；低层策略则去找一段最多 L 个 *token* 的序列 $a_t^{1:L}$ ，让它最大化一个特殊奖励——这个奖励就等于高层的价值函数 $Q^\pi(s, a_t)$ ，并且只在低层这一段 rollout 的末尾给出。

一个具体例子(网页智能体)：状态 = 至今访问过的网页交互；高层动作 = 一句话，如 `search[queen-sized bed, black]`（搜索“黑色大号床”）；奖励 = 买对了商品给 +1、否则 0；环境动态 = 网页引擎对动作的反应。而这句话里的每个 *token*（“search” “[” “queen” ……）就是低层 MDP 里的单个动作。

3.4 方法预备：RL 的几个定义

为后面公式做准备，作者复述了标准 RL 定义（用大白话解释）：

- **Q 函数** $Q^\pi(s_h, a_h)$ ：在当前步执行某动作、之后一直按策略 π 行动，能拿到的期望长期回报。形式上 $Q^\pi(s_h, a_h) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r(s_{h+t}, a_{h+t})]$ ，其中 γ 是折扣因子（越靠后的奖励越打折）。
- **价值函数** $V^\pi(s_h)$ ：把 Q 在“按策略采样的动作”上取期望，即“处在状态 s_h 的平均好坏”。
- **优势函数** $A^\pi(s_h, a_h) = Q^\pi(s_h, a_h) - V^\pi(s_h)$ ：这个动作比平均水平好多少。优势 > 0 说明这步比平均强，值得鼓励。
- 低层 MDP 的价值函数记作 $\tilde{V}(s_c, a_h^{1:i-1})$ 。

3.5 方法核心：在分层 MDP 上跑两套 RL

分层结构带来一个关键好处——两层可以各用各的算法。作者点出二者需求不同：

“只有高层需要与（不可微的）环境交互；低层只针对高层价值函数做优化，因此完全在‘体内（in silico）’训练，不需要任何环境交互。所以两层要求不同：高层算法要样本效率高，低层算法要容易优化。”

于是一个特别顺手的选择就是：**高层用 TD 学习**（off-policy，复用旧数据、样本效率高），**低层用 on-policy 策略梯度**（好优化，直接继承 RLHF 那一套）。

3.6 方法落地：面向在线 RL 的高样本效率实例

这是论文给出的可直接用的具体算法，配合图 2 看最清楚（见下一节图解）。它训两个东西：**话语级 critic**（高层）+ **token 级 actor**（低层）。

高层：话语级 critic。在高层训两个 LLM：一个表示话语级 Q 函数 $Q_\theta^\pi(s, a)$ ，一个表示话语级价值函数 $V_\psi^\pi(s)$ 。Q 模型用“延迟副本的 V 模型”算出的 Bellman 目标来训；V 模型反过来去逼近“Q 模型在 actor 自回归采样出的句子上的期望值”。因为是 off-policy，所有历史在线交互都存进数据集 $\mathcal{D} = \{s_i, a_i, r_i, s'_i\}_{i=1}^N$ 反复用。两个训练目标（公式 1、2）：

$$J_Q(\theta) = \mathbb{E}_{s,a,r,s' \sim \mathcal{D}} [(Q_\theta(s, a) - r - \gamma V_{\bar{\psi}}(s'))^2] \quad (1)$$

$$J_V(\psi) = \mathbb{E}_{s \sim \mathcal{D}} [\mathbb{E}_{a \sim \pi_\varphi(\cdot|s)} [(V_\psi(s) - Q_{\bar{\theta}}(s, a))^2]] \quad (2)$$

公式 1 逐符号拆解：让 $Q_\theta(s, a)$ 去拟合“即时奖励 r + 折扣 $\gamma \times$ 下一状态的价值 $V_{\bar{\psi}}(s')$ ”。这就是 Bellman 一致性：一个动作的价值 = 它当下拿的奖励 + 它把你带到下个局面的价值。 $\bar{\psi}$ 上面那条横杠表示延迟目标网络（target network，缓慢更新的副本），用来稳住训练、防止自己追自己发散。注意——这里的 s, a, s' 都是话语级的，所以 Bellman 回溯是一句话跨到下一句话，而不是一个 token 跨到下一个 token，时间跨度被大大缩短。

公式 2 逐符号拆解：让 $V_\psi(s)$ 去逼近“在状态 s 下，按当前 actor π_φ 采样动作 a ，再用目标 Q 模型 $Q_{\bar{\theta}}$ 打分”的期望。巧妙之处：本来话语级 TD 学习需要“在所有可能句子里取最大值”（不可行），这里改成“从 actor 采样几句话再求期望”，用 actor 产生的真实句子来近似，绕开了对天文数字动作空间求 max。两个目标网络 $Q_{\bar{\theta}}, V_{\bar{\psi}}$ 用 Polyak 平均（缓慢滑动平均）更新。

低层：token 级 actor。用 on-policy 策略梯度训 token 级策略 $\pi_\varphi(\cdot|s_c, a_t^{1:h})$ ，去找“能最大化 Q 模型预测”的 token 序列。为降低方差，用从 Q 模型导出的优势值当作末尾奖励。这个子过程其实就是把 RLHF 的单轮 RL 推广了一下，只不过末尾奖励从“奖励模型”换成了“多轮优势的估计”。用 REINFORCE 更新（公式 3）：

$$J_\varphi(\pi) = \mathbb{E}_{s_c \sim \mathcal{D}, a_t^{1:L} \sim \pi(\cdot|s_c)} \left[\sum_{i=1}^L A(s_c, a_t^{1:L}) \log \pi_\varphi(a_t^i | s_c, a_t^{1:i-1}) \right] \quad (3)$$

公式 3 逐符号拆解：对生成的每个 token a_t^i ，用“整句话的优势 $A(s_c, a_t^{1:L})$ ”去加权它的对数概率 $\log \pi_\varphi(\cdot)$ 的梯度。优势为正（这句话比平均好）就提高这串 token 的概率，为负就压低。注意优势

在句末才算出，再分摊到本句每个 token——这正是图 2 里“把优势 $A^\pi(s_c, a_t)$ 当作 token 序列末尾奖励”的含义。

3.7 方法变体：还能怎么改

ArCHer 是个框架，作者给了两个有用变体：

(1) 改进 token 级策略梯度（降方差）。当每句话很长、token 级时间跨度长时，公式 3 的 REINFORCE 方差很大、难改进策略。办法是在 token 级 MDP 里引入一个基线价值函数 $\tilde{V}_\eta$ （用蒙特卡洛回报做监督回归来训，公式 4），再把它从优势里减掉（公式 5）：

$$J_\eta(\tilde{V}) = \mathbb{E}_{s_c \sim \mathcal{D}, a_t^{1:L} \sim \pi(\cdot|s_c)} \left[\sum_{i=1}^L \left(A(s_c, a_t^{1:L}) - \tilde{V}_\eta(s_c, a_t^{1:i-1}) \right)^2 \right] \quad (4)$$

$$J_\varphi(\pi) = \mathbb{E}_{s_c \sim \mathcal{D}, a_t^{1:L} \sim \pi(\cdot|s_c)} \left[\sum_{i=1}^L \left(A(s_c, a_t^{1:L}) - \tilde{V}_\eta(s_c, a_t^{1:i-1}) \right) \log \pi_\varphi(a_t^i | s_c, a_t^{1:i-1}) \right] \quad (5)$$

直觉：基线 $\tilde{V}_\eta$ 像一条“及格线”，只奖励/惩罚超出或低于及格线的部分，让信号更干净。代价是要多训一个网络（ $\tilde{V}_\eta$ 用单独的 GPT-2 参数化），句子短时未必划算。

(2) 离线 ArCHer（完全不在线交互）。直接拿一批历史数据学。离线设定的麻烦是：改进策略往往会选到分布外动作（out-of-distribution, OOD），而这些动作的 Q 值在离线数据里估不准，硬优化会严重高估、发散、性能崩。解法是换目标函数：用 **IQL**（implicit Q-learning，隐式 Q 学习）给 critic 算 Bellman 目标（只在“数据支持的动作”上做），用 **AWR**（advantage-weighted regression，优势加权回归）约束 actor 别离数据太远。

IQL 损失（公式 6）：用一个期望分位数（*expectile*）回归来隐式地近似“对动作取最大值”，而不用真的去 max：

$$J_\psi^{\text{IQL}}(V) = \mathbb{E}_{s \sim \mathcal{D}} \left[\mathbb{E}_{a \sim \pi_\varphi(\cdot|s)} [L_2^\tau(V_\psi(s) - Q_{\tilde{\theta}}(s, a))] \right] \quad (6)$$

其中 $L_2^\tau(u) = |\tau - \mathbf{1}\{u < 0\}|u^2$ ， $\tau \in [0.5, 1)$ 是期望分位数参数（ τ 越大越偏向“乐观的高值”，即越接近取 max）。

AWR 损失（公式 7）：在“追高回报”和“模仿生成数据的策略”之间折中：

$$J_\varphi(\pi) = -\mathbb{E}_{(s_c, a_t^{1:L}) \sim \mathcal{D}} \left[\exp(\beta \cdot A(s_c, a_t^{1:L})) \sum_{i=1}^L \log \pi_\varphi(a_t^i | s_c, a_t^{1:i-1}) \right] \quad (7)$$

β 是用户设的正标量： β 小 $\rightarrow$ 偏向模仿原数据（稳）； β 大 $\rightarrow$ 更激进地追奖励（可能不稳）。

3.8 框架总结与实现细节

伪代码（原文 Algorithm 1）的脉络：每轮迭代里——(a) 在线模式下用当前 actor 收集数据进缓冲区；(b) 用 Bellman 自举更新话语级 Q 、 V （公式 1 与公式 2 或 6），并用滑动平均更新目标网络；

(c) (可选) 用蒙特卡洛回归更新 token 级基线 (公式 4); (d) 用话语级 critic 给的优势更新 token 级 actor (公式 3、5 或 7)。同一套伪代码既能跑在线也能跑离线, 且高/低层目标可灵活替换——这正是“框架”的意义。

关键实现选择: 策略 (actor) 用 **GPT-2** 架构; critic 用 **RoBERTa-base** (取 “[CLS]” token 的嵌入接一层线性层出预测)。为对抗 Q 值高估, 用双 Q 学习 (double Q-learning) trick: 独立训两套 $\{Q_1, V_1\}$ 、 $\{Q_2, V_2\}$, 优势用 Q_1, Q_2 取最小、 V_1, V_2 取最小来算 (取 min 更保守, 防高估)。为省算力/显存, 四个网络共享同一个语言模型编码器骨干, 只用各自独立的 MLP 头; actor 的参数与 critic 独立。

3.9 理论分析：为什么“分层”在理论上也更优

作者证明了分层设计相比 token 级 off-policy 方法, 对算法收敛的条件要求更弱, 且有限样本下的统计误差更小。两点直觉 (对应 Lemma 1、2):

- **Bellman 完备性 (函数类容量):** 要满足 Bellman 完备这个条件, 话语级 critic 的函数类要求更弱。直觉是 token 级 critic 必须能在“下一个 token”这个细粒度上拟合任意函数 (要求容量很高), 而话语级 critic 只需在“话语”这个更粗的时间尺度上拟合任意函数。
- **数据覆盖 (密度比条件):** 话语级和 token 级要求一样。直觉是同一批离线数据在两层诱导出的轨迹分布是一样的——一条轨迹在话语级被覆盖, 在 token 级也被覆盖, 反之亦然。

主定理 (Theorem 1, 非正式) 的结论: 在最坏情况下, 用 token 级 critic 估计优势的误差, 比用话语级 critic 大约大 $\sqrt{\gamma^L}$ 倍 (L 是每句话最多的 token 数), 原因是误差累积。实践里 γ 常 > 0.95 、 L 可达 64, 导致 $\gamma^{L^{1/2}}$ 这个因子远大于 1。换句话说: 分层设计不仅收敛条件更弱, 统计误差保证也更好, 最终给出的策略梯度估计在最坏情况下更准。

4 方法图解：ArCHer 的架构与数据流

图 1 是 ArCHer 实用版的整体示意 (原文 Figure 2), 上下分两层, 是理解全文的关键。

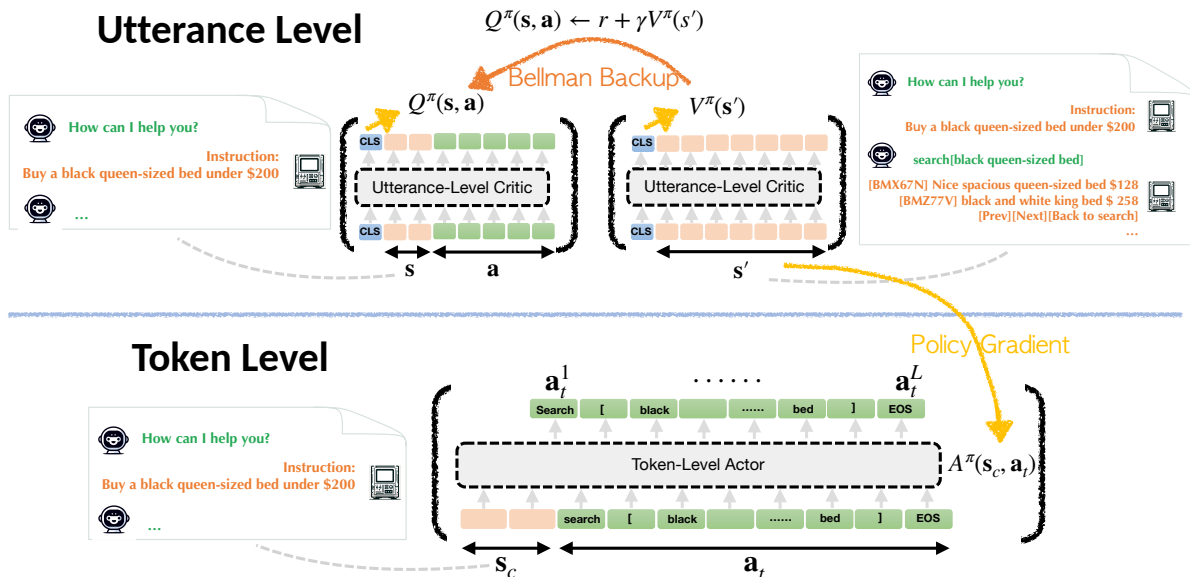


图 1: ArCHer (带分层结构的演员-评论家框架) 实用实例示意。算法同时在话语级与 *token* 级运作。在话语级, 通过带 TD 误差的 Bellman 自举学习一个 Q 函数; 在 token 级, 用策略梯度法最大化“由话语级 Q 函数诱导的优势函数”, 并把这个优势估计作为本句 token 序列末尾的奖励。(图内英文为原文保留。)

怎么读这张图:

- **上半部分 (Utterance Level, 话语级):** 把一整句话当作一个动作。左边的 Utterance-Level Critic 吃进“状态 s + 动作 a (一句话)”, 输出 $Q^\pi(s, a)$; 右边同一个 critic 吃进“下一状态 s' ”, 输出 $V^\pi(s')$ 。中间橙色箭头 **Bellman Backup** (Bellman 回溯) 执行公式 1: $Q^\pi(s, a) \leftarrow r + \gamma V^\pi(s')$ ——注意箭头是一句话跨到下一句话, 所以时间跨度短。这一层用 off-policy 数据训, 样本效率高。图中那个“买黑色大号床”的例子展示了状态就是“与购物网站来回的交互历史”, 动作就是 `search[black queen-sized bed]` 这样的一句话。
- **下半部分 (Token Level, token 级):** 把这句话里的每个字当作一个动作。Token-Level Actor 在“本轮上下文 s_c ”基础上, 一个字一个字地自回归生成 $a_t^1, \dots, a_t^L$ (如“Search” “[” “black” “...” “bed” “]” “EOS”)。
- **两层的连接 (橙黄色 Policy Gradient 大箭头):** 话语级 critic 算出的优势 $A^\pi(s_c, a_t)$, 被当作这句话末尾的奖励, 顺着大箭头“喂”给下层的 token 级 actor, 用策略梯度 (公式 3/5) 更新它。这就是“高层评委打分、低层据此学怎么说话”的数据流。

一句话概括这张图: 高层 (话语级) 当评委、用能复用旧数据的 *TD* 学习快速学会“评估一句话的好坏”; 低层 (*token* 级) 当演员、用 *PPO* 那套 *on-policy* 梯度、拿评委给的优势分当奖励, 学会“具体每个字怎么生成”。

5 实验解读

5.1 用了哪些任务、比了哪些基线

评测任务 (5 个)。作者特意挑了需要多步规划、不能一轮解决、7B 内模型能做、且评测快而可复现的任务：

- **Detective Game** (侦探游戏)：交互式文字冒险，要靠环境反馈一步步生成自然语言动作（如“take paper”），最终目标是找出凶手；到达里程碑给奖励。
- **Twenty Questions** (二十问)：智能体当“猜词者”，在 20 个是/否问题内从 157 个词里猜出隐藏词。神谕 (oracle) 用 `flan-t5-small` 模拟。猜对得 0、终止；否则每步 -1。还有一个只含 10 个词的简化版 **Twenty Questions Subset** (考“走捷径”的专门策略)。
- **Guess My City** (猜城市)：从 100 个城市里 20 问内猜出隐藏城市。与二十问的关键区别：可以问任意问题、答案是自由文本而非只有是/否（更长更难）。
- **WebShop**：工具使用任务，按指令去购物服务器上买东西，按“买到的商品与要求的相似度”给 0~1 的稠密奖励。

对比基线：

- **token 级 PPO**：当前 LLM 上的 SOTA on-policy 方法，但旧数据用完即弃。
- **Filtered BC** (过滤式行为克隆)：非 RL 基线，只模仿最近 rollout 中回报最高的前 10%。作者强调：这个简单基线常常出奇地强，所以“能否超过 Filtered BC”是判断一个方法里“RL 成分”是否真正起作用的试金石。
- **CHAI**：SOTA 话语级 RL 方法。关键区别——CHAI 用一个冻结的 actor（行为克隆得到）采样 k 个候选句子，再让 critic 挑 Q 值最高的；而 ArCHer 会一起优化 *actor*。实验取 $k = 5$ （即便如此，CHAI 运行时间已是 ArCHer 的约 4 倍）。
- 在 WebShop 上还比了 **GPT-3.5 + 专家提示**和 **GPT-3.5 + ReAct**。

所有方法的 token 级策略都用“在次优数据上做监督指令微调”的同一个 checkpoint 初始化，保证公平、且利于 RL 初期探索。

5.2 主要结果：100 倍样本效率 + 更强最终性能

表 1: 核心结论速览（依据原文图 3、图 4、5.3 节文字整理）。

对比对象	结论
vs. PPO (on-policy)	在 4 个较难任务上 ArCHer 收敛到明显更好的性能；样本效率约 100 倍 ：二十问上 PPO 要 >10 万样本才让平均回报勉强高于 -17，ArCHer 不到 1000 样本就达到同一水平。
vs. Filtered BC	Filtered BC 收敛很快但停在次优、给更多数据也不再提升；ArCHer 随样本增多持续稳定改进。
vs. CHAI (话语级 RL)	CHAI 初期比 ArCHer 快，但最终收敛得更差（因为它只在冻结策略上重排候选，改进空间窄）；ArCHer 需要一段初期学习把 critic 误差降下来，之后稳步超越。
WebShop	用 ArCHer 微调的 <i>GPT-2</i> ，是唯一超过“GPT-3.5 + 专家提示 / ReAct”的方法。

整体上：ArCHer 在 4 个更难的任务（Twenty Questions Subset、Twenty Questions、Guess My City、WebShop）上收敛到比所有前人方法都好的性能；在最简单的 Detective Game 上则与最佳前人方法打平。

5.3 消融实验：拆开看每个设计是否有用

- **off-policy 数据的重要性** (图 5b, Guess My City): 把回放缓冲区调小 (只留最近 48 条 rollout) → 学习不稳定 (在有限数据上过拟合)；缓冲区大一些更稳，但大到一定程度后再增大就没明显收益。结论：用好 off-policy 数据能提升稳定性与性能。
- **critic 用什么骨干** (图 5c, Twenty Questions): 把 critic 从仅编码器的 RoBERTa 换成仅解码器自回归的 GPT-2 (取最后 “[EOS]” token 嵌入)。结果：RoBERTa 版开头略快，但过了初期两者表现一致。结论：ArCHer 用解码器模型当 critic 也行，不掉性能。
- **模型规模 100M→7B** (图 5d, Twenty Questions Subset): 把 token 级 actor 从 1 亿参数的 GPT-2 换成 70 亿的 **Mistral-7B** (7B 开源 checkpoint 零样本已有不俗表现，无需再做监督微调)。结果：Mistral-7B 版学得快得多。结论：ArCHer 能随模型变大而更好地 *scale*——这与“actor 更新本质上和 RLHF 单轮微调相似、应同样受益于规模”的预期一致。
- **token 级基线** (图 5a, Guess My City): 这个任务句子长而多样。加上 token 级基线 (公式 5) 大幅超过不加基线的标准 ASCHer，验证了“基线能在长/多样句子时有效降方差”的假设；代价是要多训一个 $\tilde{V}_\eta$ ，句子短时未必值。

5.4 离线设定: IQL + AWR 最稳 (表 1)

表 2: ArCHer 在离线 RL 设定下的不同变体 (依据原文 Table 1, Twenty Questions 上的回报 Return, 跑 1280 条轨迹 / 5 个随机种子)。

方法 (离线)	Twenty Questions (Return)
ArCHer (IQL + AWR)	-14.1
ArCHer (IQL + REINFORCE)	-20
ArCHer (IQL + REINFORCE + BC)	-15.3
ArCHer (SARSA + AWR)	-14.5
Filtered BC	-15.4
BC	-16.8

读这张表 (回报越接近 0 越好):

- 直接把在线的 REINFORCE (公式 3) 搬到离线 → 性能迅速崩塌 (-20), 因为没有任何约束防分布外动作, 这是离线 RL 的经典坑。
- 给它加上模仿损失 (+BC) 能稳住、提升到 -15.3, 但仍不如 AWR。
- 把 IQL 的“数据内分位数回溯”换成 **SARSA** 回溯 (直接用数据里下一句话算 Bellman 目标、不做任何取最大值) → -14.5, 不如 IQL (-14.1)。这凸显了离线设定下“对动作取最大值”的重要性。
- **结论:** 用 IQL + AWR (best, -14.1) 这种专门处理分布外动作的组合最好; 且 IQL/SARSA 配 AWR 都优于朴素 BC 和 Filtered BC, 说明“动态规划训 critic”确实有用。

6 核心 takeaways (给 AI 应用开发者)

1. **“分层”是处理多轮长程任务的关键招式。** 当你的任务是“模型要来回好几轮才能完成” (客服、网页操作、工具链、对话搜集需求), 不要硬套单轮 RLHF。ArCHer 的思路——高层管“说哪句话”、低层管“这句话怎么生成”——把“超长时间跨度”和“超大动作空间”这对矛盾一次性化解。
2. **样本效率 = 真金白银。** 多轮任务里每次与环境交互 (真去点网页、真调 API、真等用户回复) 都慢且贵。能复用历史数据的 off-policy 设计带来约 100 倍样本效率 (相对 PPO), 意味着同样的预算下少跑两个数量级的交互。做 agent 训练时, 优先考虑能复用 *replay buffer* 的方案。
3. **“能否打败 Filtered BC”是检验 RL 是否真起作用的试金石。** 过滤式行为克隆 (只模仿历史最好的前 10%) 简单到几乎是免费基线, 却常常很强。你自研的 RL 方案如果打不过它, 多半是“RL 成分”没真正生效——这是个极实用的 sanity check。

4. **在线 / 离线要用不同的“配方”**。在线可用简单的 REINFORCE；但一旦改成从静态历史数据离线学，必须换成能压制分布外动作的目标 (IQL 给 critic、AWR 给 actor)，否则会高估 Q 值、训练发散。把在线算法原样搬到离线是常见翻车点。
5. **这套框架对模型/算法是“即插即用”的**。actor、critic 的骨干可换 (GPT-2、RoBERTa、Mistral-7B 都行)，高/低层算法可换 (TD / IQL / SARSA、REINFORCE / AWR)。而且 actor 的更新本质上就是 RLHF 那一套，能直接复用现有单轮 RL 的工程实现，并随模型变大而 scale。
6. **已知边界 (用前要知道)**。论文主实验多用较小的 GPT-2；虽有 Mistral-7B 的良好 scaling 迹象，但大模型 + 更多基准的系统验证仍是未来工作。此外方法仍需数千次交互、且只在“可计算的奖励”任务上验证——若你的场景只有约 100 次真人交互的预算，作者认为基于模型的 RL (model-based RL) 可能更有前途。

参考文献 (仅列与本篇最相关的几条，英文原样保留)

1. Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, Aviral Kumar. ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL. ICML, 2024. <https://github.com/YifeiZhou02/ArCHer>
2. Charlie Snell et al. Offline RL for Natural Language Generation with Implicit Language Q Learning (ILQL). ICLR, 2023.
3. Siddharth Verma et al. CHAI: A Chatbot AI for Task-Oriented Dialogue with Offline RL. NAACL, 2022.
4. Ilya Kostrikov, Ashvin Nair, Sergey Levine. Offline Reinforcement Learning with Implicit Q-Learning (IQL). ICLR, 2022.
5. Xue Bin Peng et al. Advantage-Weighted Regression (AWR): Simple and Scalable Off-Policy Reinforcement Learning, 2019.
6. John Schulman et al. Proximal Policy Optimization Algorithms (PPO). arXiv:1707.06347, 2017.
7. Marwa Abdulhai et al. LMRL Gym: Benchmarks for Multi-Turn Reinforcement Learning with Language Models, 2023.